

# Hospital Management System

Object-Oriented Programming — Semester Project (C++)

Submitted To: Engr. Faisal Zia

Group Members:

Muhammad Sibtain (062)

Mamoon Asif (034)

Rumesa Jamil (072)



# Introduction

A Hospital Management System (HMS) is a software solution that centralizes clinical and administrative tasks — patient records, appointments, staff details, and reporting. Digital HMS improves accuracy, speed, and coordination across departments.



## Why it matters

Reduces errors, speeds workflows, and enables data-driven decisions for patient care.



## Project scope

Console-based C++ implementation demonstrating core HMS functions and OOP principles.





# Problem Statement

- Manual systems cause data duplication, misplaced files, and slow retrieval.
- Managing patient and doctor records manually is error-prone and time-consuming.
- Appointment scheduling lacks coordination and leads to conflicts and long waits.
- There is a clear need for automation, persistent storage, and secure access control.

# Project Objectives

## Manage Patients

Create, search, update, and delete patient records.

## Manage Doctors

Store doctor profiles, specializations, and availability.

## Automate Appointments

Book, view, and cancel appointments with conflict checks.

## Persistent Storage

Save data using file handling to create durable records.

## OOP in C++

Apply encapsulation, inheritance, polymorphism, and abstraction.

# System Features

The image displays two screenshots of a Hospital Management System interface.

**Left Screenshot: Hospital Management System - Login**

- Username:** admin
- Password:** \*\*\*\*\*
- Login** button
- Exit** button

**Right Screenshot: Hospital Management System**

- Add Patient** button
- View Patients** button
- Search Patient** button
- Delete Patient** button
- Add Doctor** button
- View Doctors** button
- Search Doctor** button
- Book Appointment** button
- View Appointments** button

Add / Search / Update / Delete Patients

Add / Search Doctors

Book & Cancel Appointments with conflict validation

Login & basic access control

Hospital reports: daily appointments, patient lists

File handling for persistent storage

# OOP Concepts Applied



## Encapsulation

Classes encapsulate data and provide public methods for controlled access (getters/setters).



## Inheritance

Base class Person  
→ derived classes Patient and Doctor to reuse attributes and behavior.



## Polymorphism

Virtual functions allow different behaviors for shared interfaces (overrides for display, save).



## Abstraction

Expose high-level operations (bookAppointment, saveRecord) while hiding file I/O details.

# Class Diagram — System Design

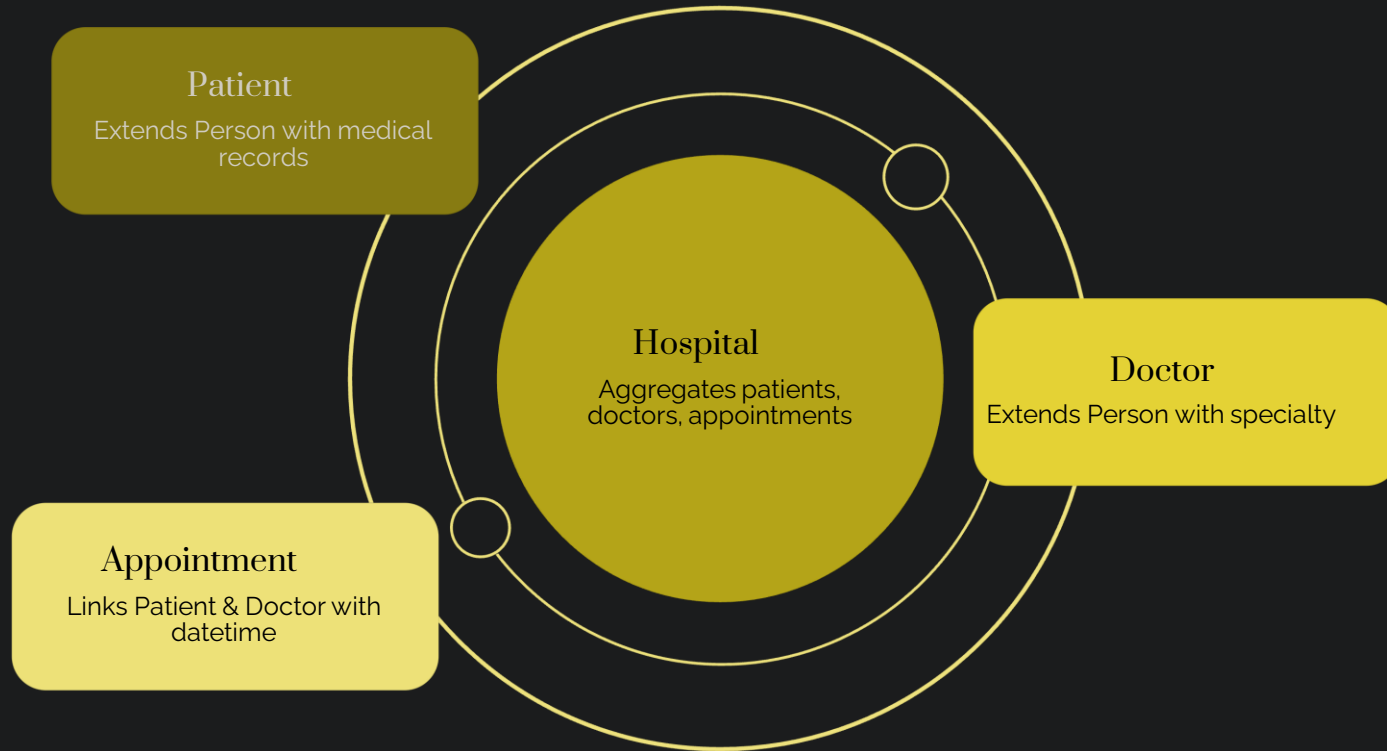


Diagram notes: Person is parent class. Patient/Doctor extend Person. Appointment associates Patient & Doctor (composition). Hospital aggregates all domain entities and provides management methods (addPatient, scheduleAppointment, generateReport).

# File Handling & Data Storage

Persistent storage uses plain text files for simplicity and portability. Data saved and loaded during program start/exit to maintain state across runs.

- patients.txt — patient records (ID, name, contact, history)
- doctors.txt — doctor profiles (ID, name, specialization, schedule)
- appointments.txt — appointment entries (patientID, doctorID, datetime, status)





# Advantages & Future Improvements

## Advantages

- Faster administrative workflows and reduced waiting times
- Less paperwork and lower risk of lost records
- Quick search and filtered retrieval of records
- Organized, auditable patient and appointment history

## Future Improvements

- GUI using Qt for an intuitive desktop interface
- Database integration (MySQL / SQLite) for scalable data management
- Billing & invoicing module with configurable tariffs
- Online appointments, notifications, and role-based access control

THANK YOU  
*HIT & CAECUR*



## Conclusion & Thank You

The Hospital Management System demonstrates practical application of OOP in C++: modular class design, file-based persistence, and essential features for clinical administration. This project offers a foundation for GUI and database enhancements toward a production-ready system.

### Practical Outcomes

Working console app with CRUD operations, appointment handling, and persistent storage.

### Q&A

Questions & Answers